

传输层概述

应用程序 文件 FTP TELNET 邮件 SMTP 网址→IP地址 DNS TFTP HTTP SNMP
熟知端口号 21 23 25 53 69 80 161

端口的作用

通过“端口号”标识本主机的一个特定进程

注意：每台主机的端口号相互独立

注意：TCP、UDP两种协议的端口号相互独立

TCP或UDP协议，通过Socket套接字={IP地址:端口号}唯一地标识网络中一台主机上一个应用程序

端口号的分类

服务器使用的端口号

熟知端口号 0~1023 (通常只用于熟知应用程序)

登记端口号 1024~49151

客户端使用的端口号：短暂端口号 49152~65535

功能

实现端到端(进程到进程)的通信

复用和分用

复用(从上到下)：发送数据时同一台主机上的多个进程可使用同一个传输层协议

分用(从下到上)：接收数据时传输层可以把数据正确交付到目的进程

差错检测

TCP检测出差错后丢弃数据并通知发送方重传

UDP检测出错误后直接丢弃数据，不通知发送方

向应用层提供两种服务

传文件
看直播

传输前先打招呼，确认对方已准备好接收数据，结束时告知对方已结束

面向连接的、可靠的端到端传输服务(TCP)——确保数据正确

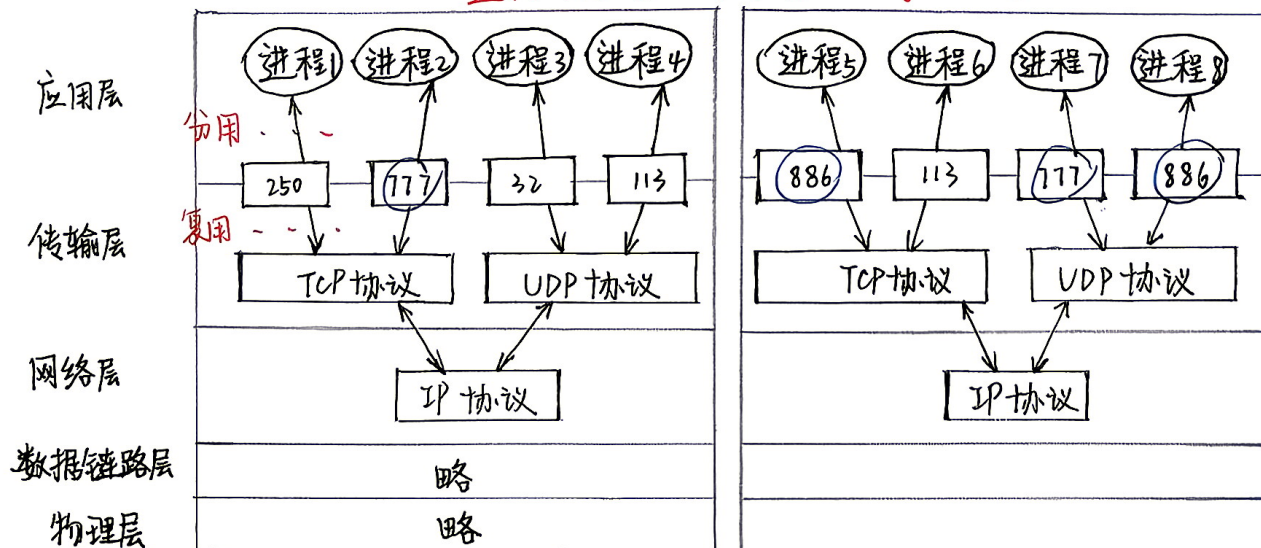
无连接的、不可靠的端到端传输服务(UDP)——数据可能出错/丢失，但开销小、速度快

不提前打招呼，直接传数据

一个IP地址可以标记网络中一台特定的主机，网络层可以在IP数据报首部添加“源IP地址”、“目的IP地址”这2个信息来实现“主机到主机”的通信

一个IP数据报到达主机之后，其所携带的数据还要交给上面某一应用进程，此时传输层的端口号会发生作用，每台主机上的一个进程绑定一个端口号(不重复)，用主机IP地址和该主机IP地址上某一特定进程的端口号就可以精准指向网络中某一主机的某一进程，传输层通过端口号来区分数据来自去往哪个进程，在TCP或UDP报文段的首部指明“源端口”、“目的端口”这2个信息来实现“端到端”(“进程到进程”)的通信

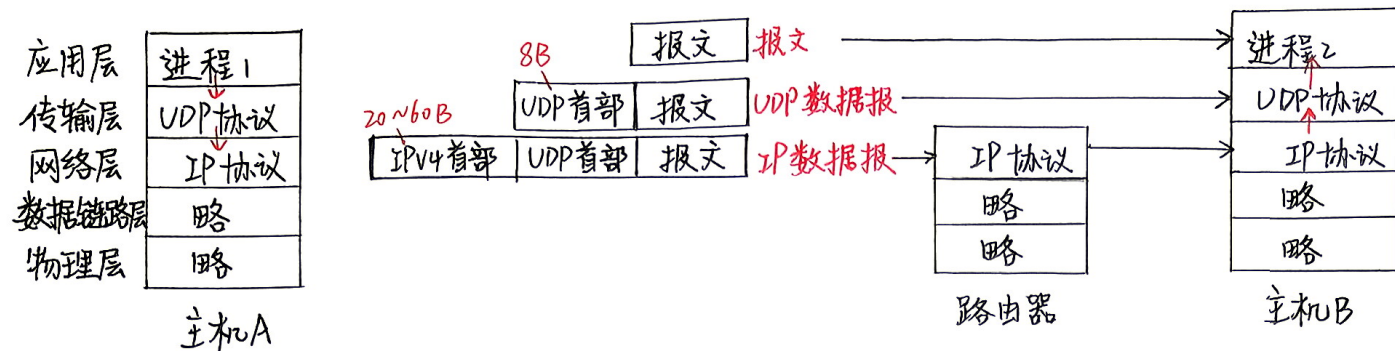
进程、端口号、传输层协议之间的关系



两台主机的端口号相互独立。
TCP、UDP的端口号相互独立。
当两个进程之间想通信时，需指明：
① 使用何传输协议
② 本进程绑定的端口号
③ 对方的IP地址和端口号

↓
套接字(Socket)
{IP地址:端口号}

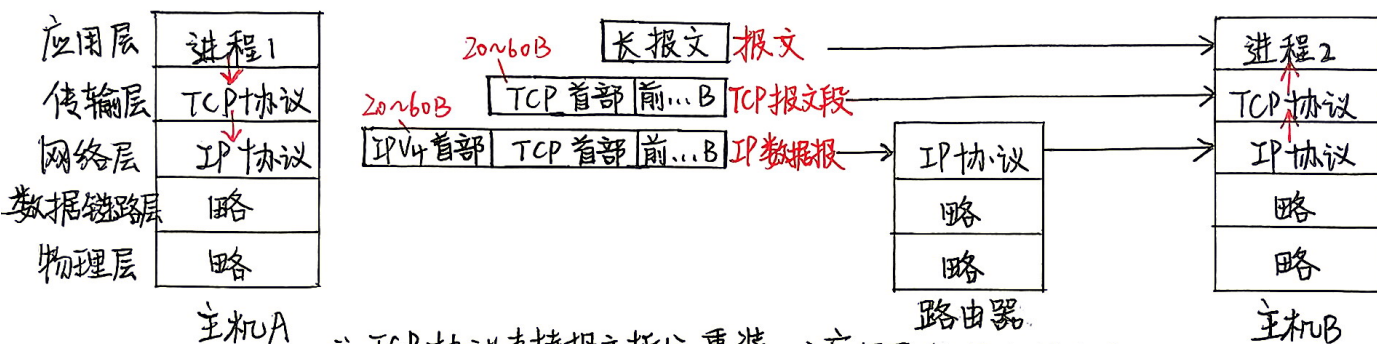
UDP 协议、UDP 数据报、UDP 首部



∴ UDP 协议不支持报文拆分、重装：应用层的报文长度不能超过 UDP 上限

- UDP 首部很小，只占 8B
- UDP 每次传输一个完整报文，不支持报文自动拆分、重装
- UDP 是无连接的、不可靠的（可靠性可以交给应用层处理），不支持拥塞控制
- UDP 支持一对一（封装成单播 IP 数据报）、一对多（封装成广播 IP 数据报）传输

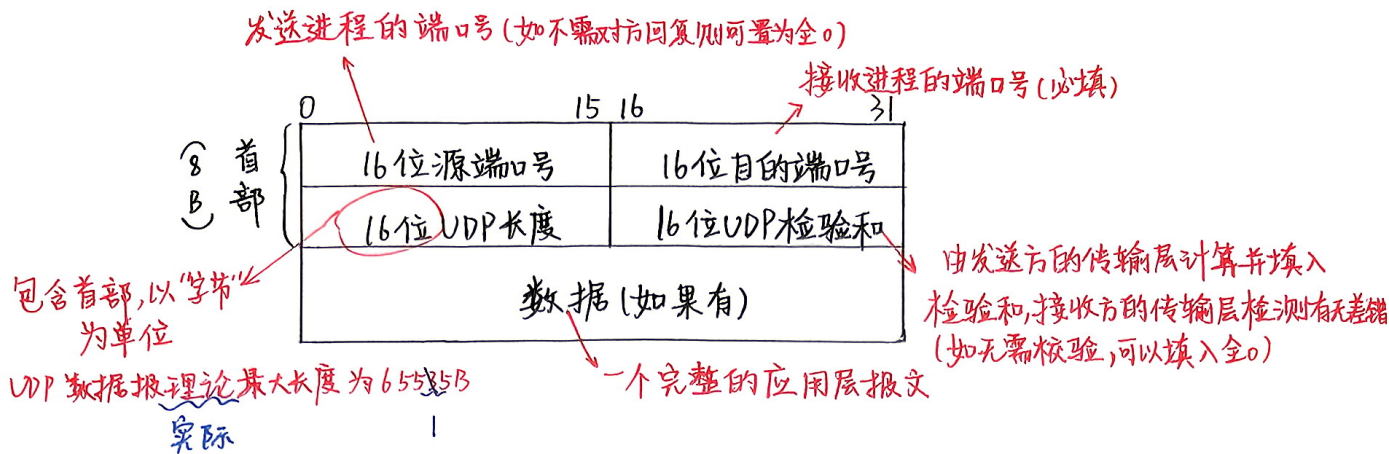
TCP 协议、TCP 报文段、TCP 首部



∴ TCP 协议支持报文拆分、重装：应用层的报文较长也是可以的

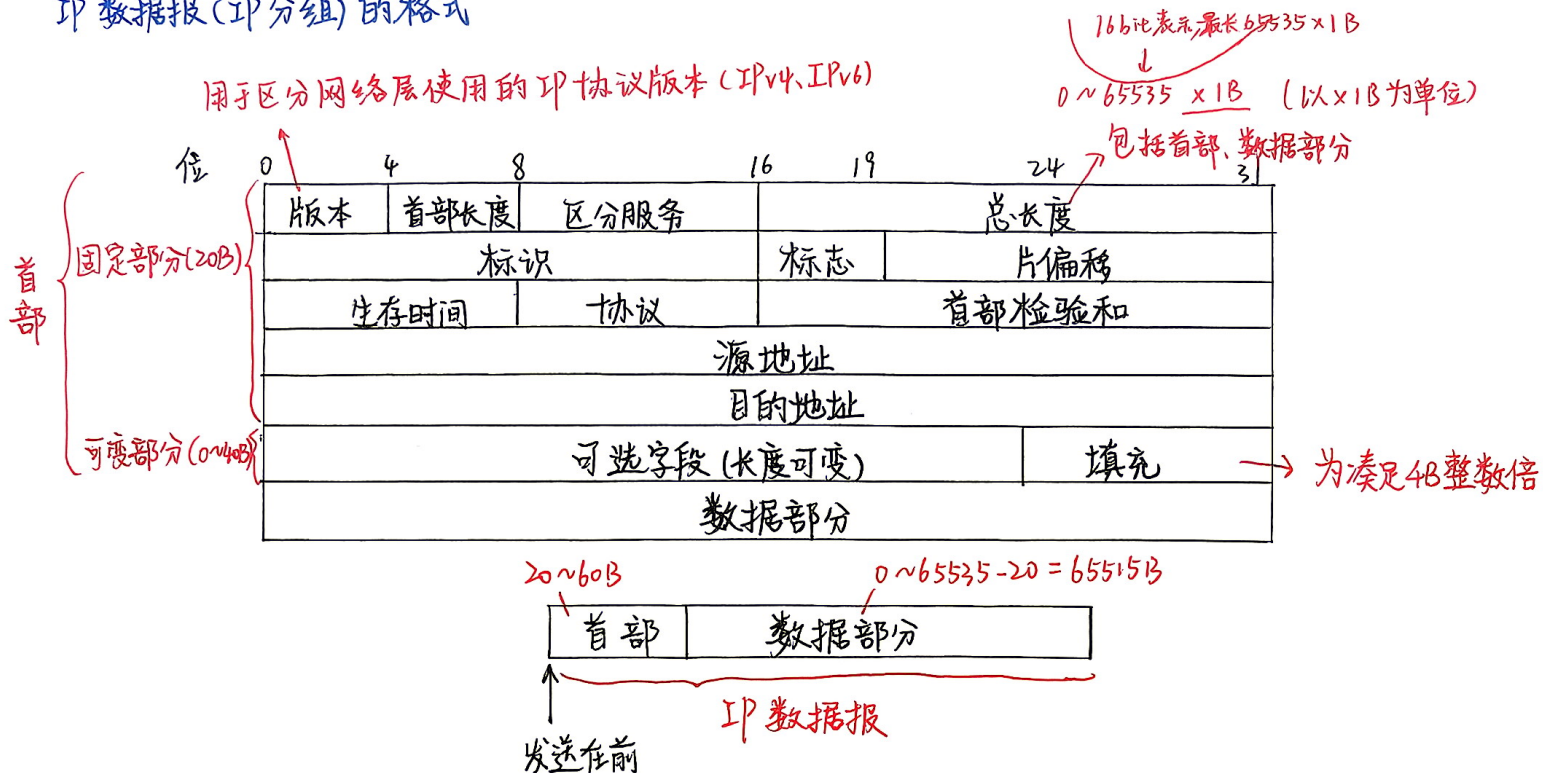
- TCP 首部更大，占 20~60B
- TCP 支持报文自动拆分、重装，因此可以传输长报文
- TCP 是有连接的、可靠的、支持拥塞控制
- TCP 仅支持一对一传输，因为通信双方的传输层必须先建立连接

UDP 数据报格式

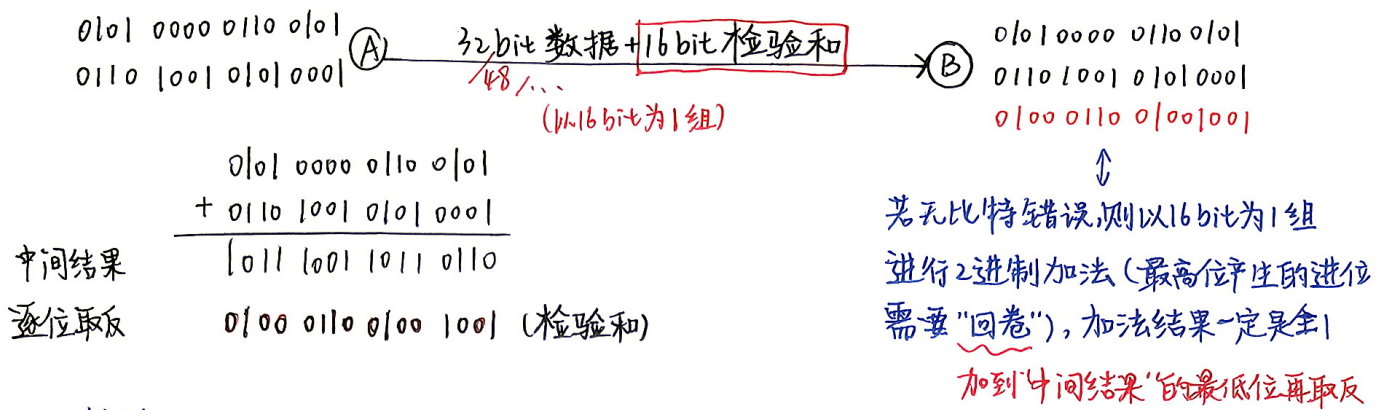


IP 数据报 (IP 分组) 的格式

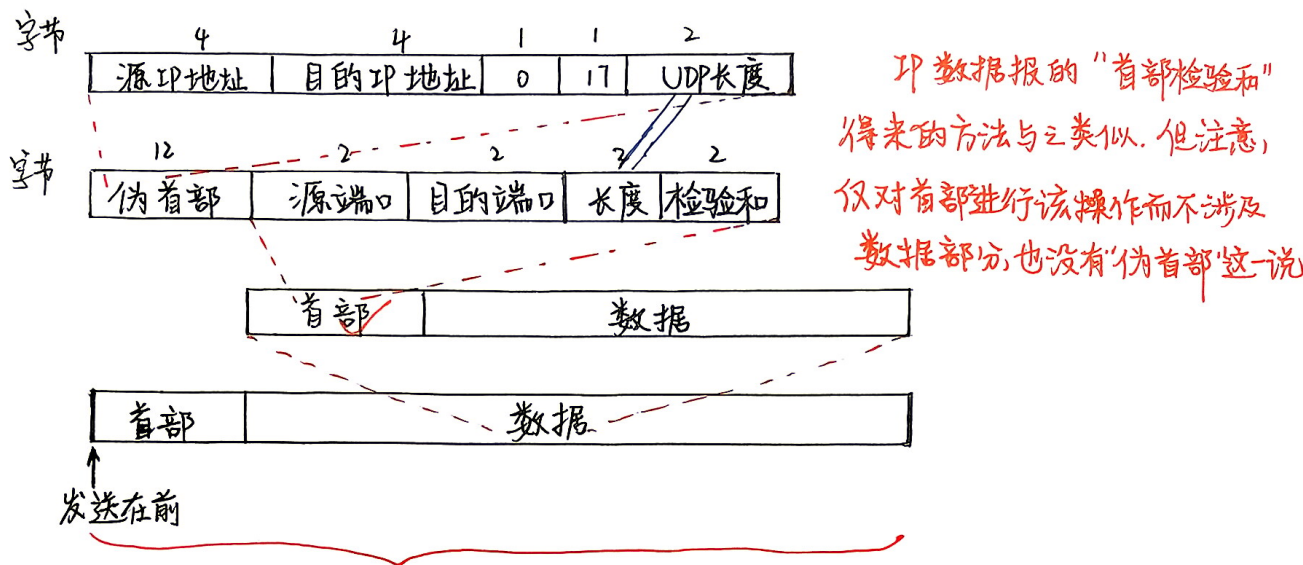
用于区分网络层使用的IP协议版本 (IPv4, IPv6)



一种新的差错检验方法



UDP 校验



发送方的传输层:

IP 数据报

- 传输层的UDP协议计算校验和之前,先添加伪首部
- 把伪首部、首部、数据部分以16 bit为一组,进行2进制加法(最高位产生的进位要回卷)
- 把最终的加法结果逐位取反,得16 bit校验和,将其填入UDP首部
- 去掉伪首部,将UDP数据报交给网络层,封装成IP数据报

接收方的传输层:

- 网络层向传输层递交UDP数据报
- 传输层在UDP数据报前添加伪首部
- 把伪首部、UDP首部、数据部分以16 bit为1组进行2进制加法(最高位产生的进位要回卷)
- 若加法结果为全1,说明没有比特错误,接收该UDP数据报,并根据目的端口号向应用层递交报文

若加法结果不是全1,说明有差错,丢弃该UDP数据报

例. 若UDP协议在计算校验和过程中,计算得到中间结果为1011100110110110时,还需加上最后个16位数0110010111000101,最终计算得到的校验和是:

$$\begin{array}{r} 1011\ 1001\ 1011\ 0110 \\ + 0110\ 0101\ 1100\ 0101 \\ \hline 10001\ 1111\ 0111\ 1011 \\ + \\ \hline 10001\ 1111\ 0111\ 1011 \end{array}$$

↓ 取反

$$0001\ 1111\ 0111\ 1100$$

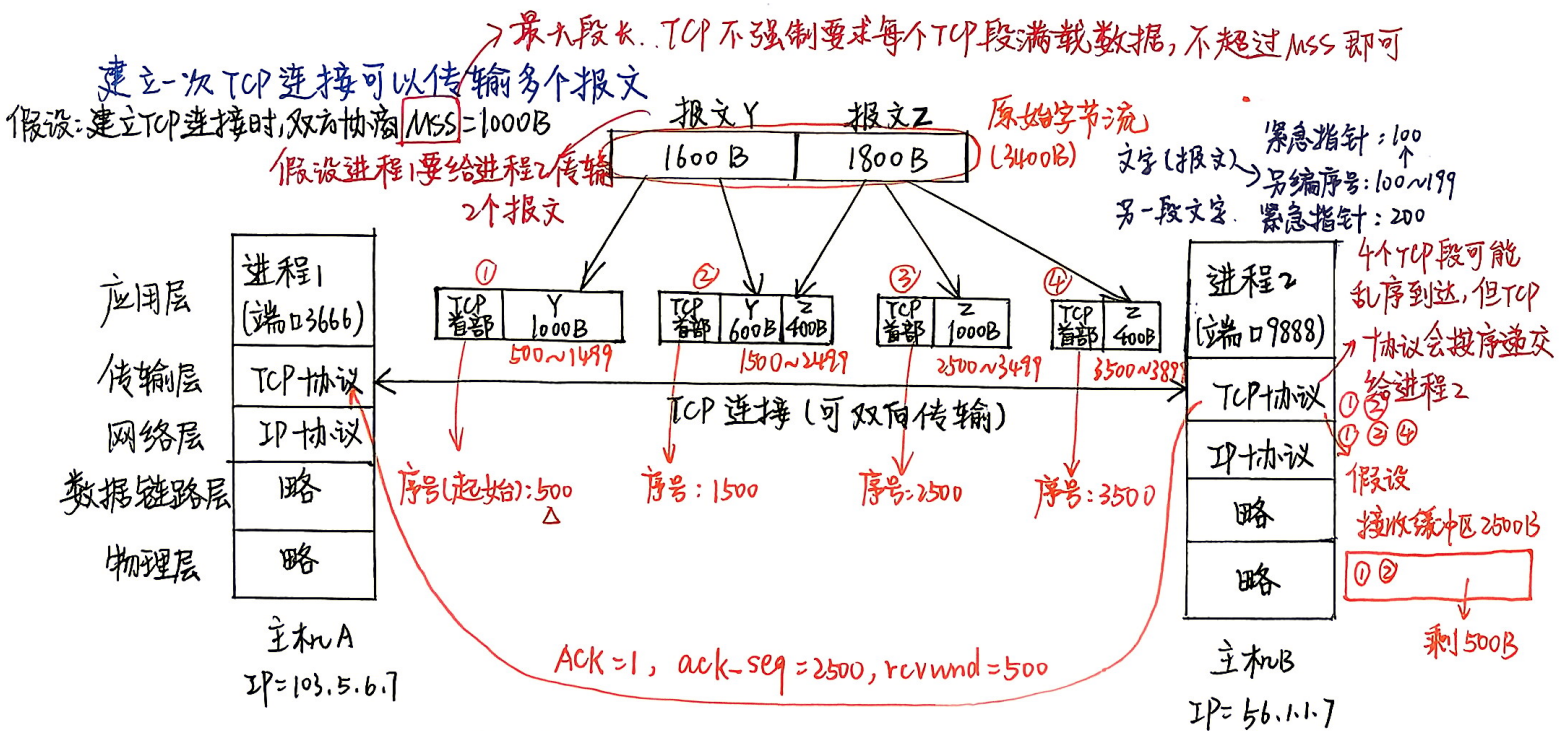
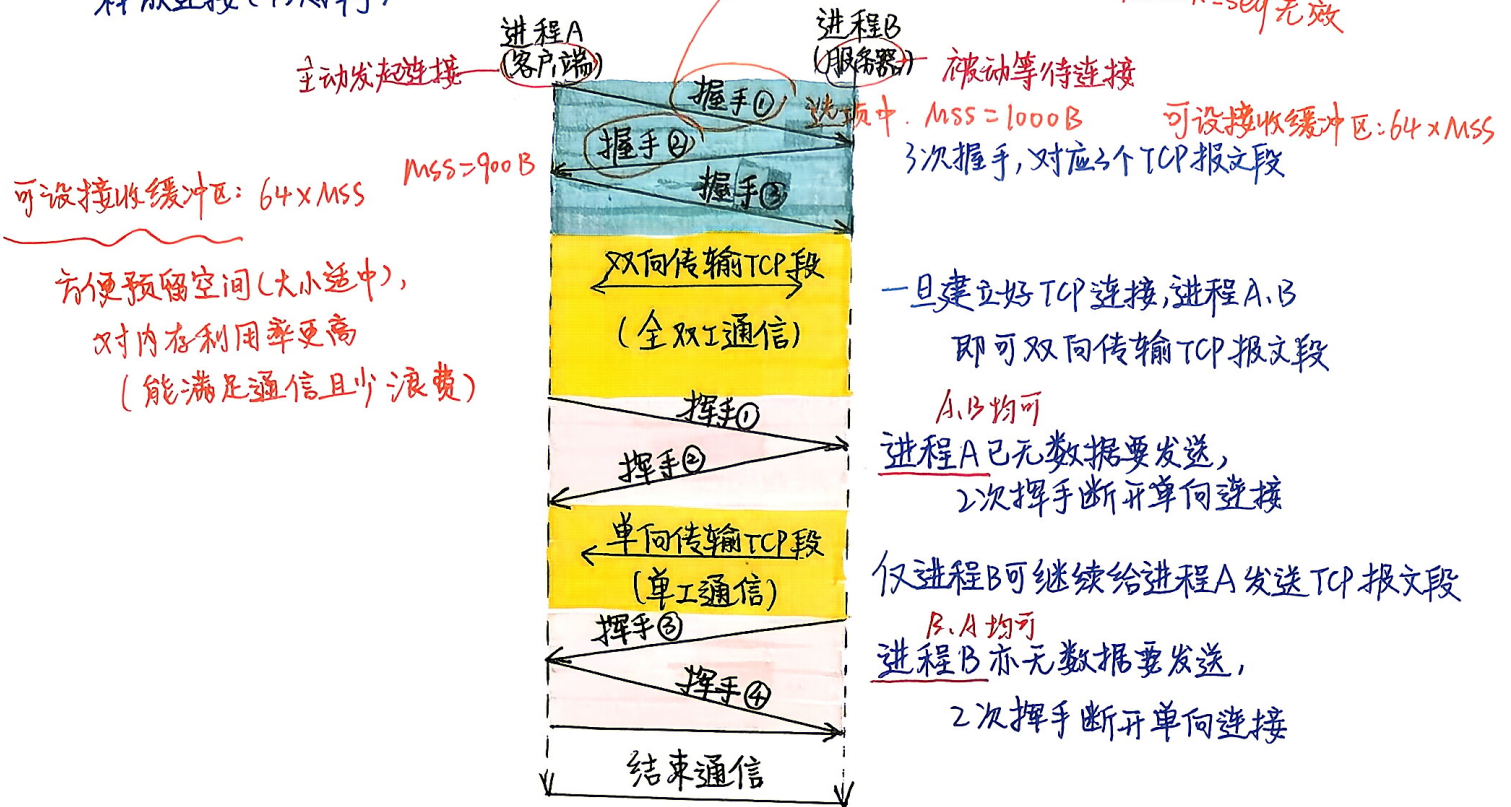
$$1110\ 0000\ 1000\ 0011$$

TCP协议的三大阶段

建立连接(3次握手)

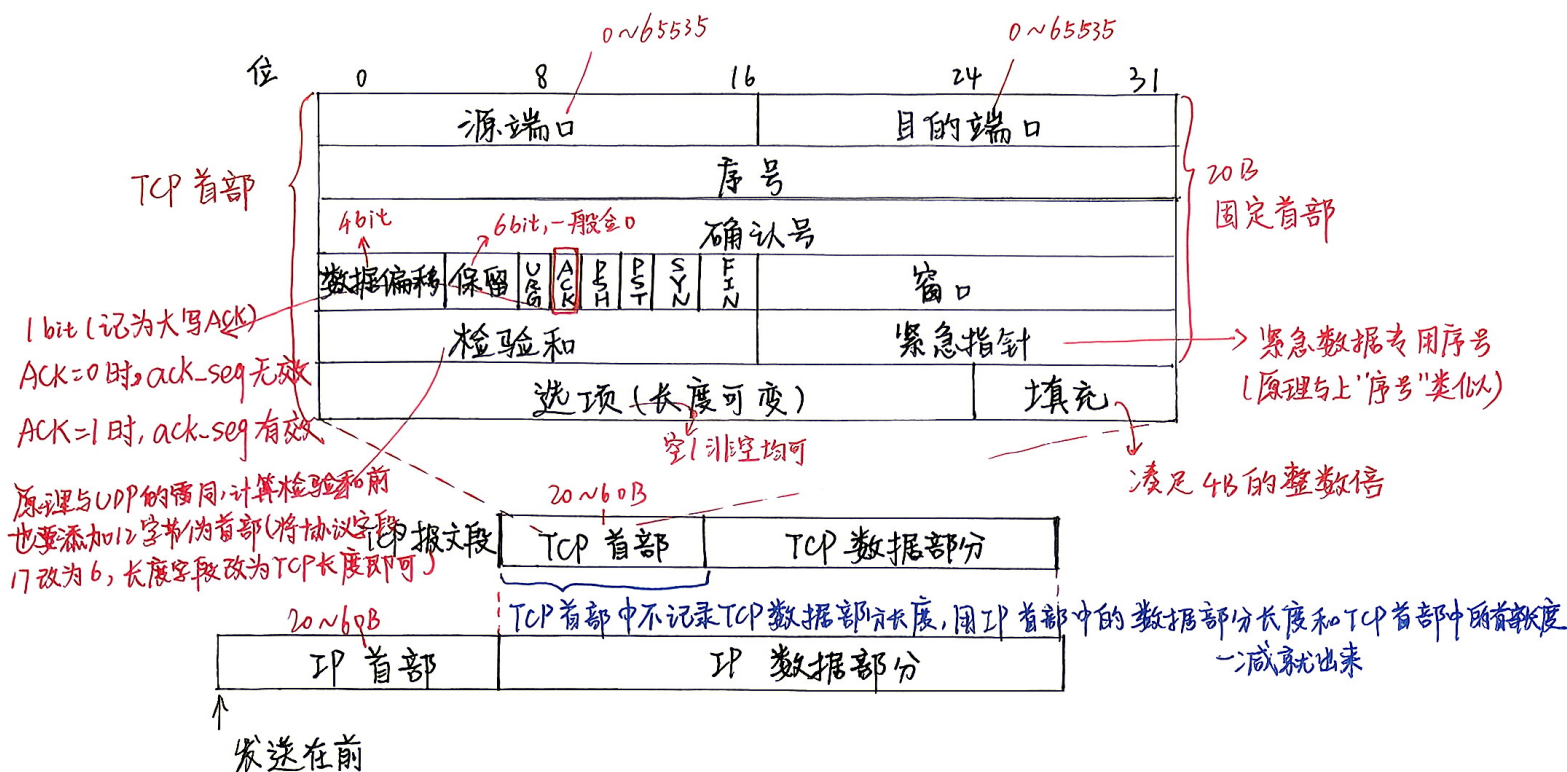
数据传输

释放连接(4次挥手)



- 每次建立TCP连接, 可双向传输多个报文
- TCP面向字节流而UDP面向报文, 不论传输多少报文, 在TCP协议看来都是一连串字节流

TCP段格式



序号 (seq) 用于标记数据部分第1个字节在原始字节流中的位置

确认号 (ack或ack-seq) 有“累计确认”的作用, 用于反馈, 表示序号在该确认号之前的所有字节都已正确收到

- 起始“序号”是发送方自己设置的, 不一定从0开始
- 只有握手①的ACK=0, 其它所有TCP报文段的ACK=1

数据偏移 (Data Offset) 表示TCP首部长度的, 以 $\times 4B$ 为单位
 $0 \sim 15 \times 4B \Rightarrow$ 最长: 60B
 固定: 20B

“xxx段”即表示该标志位为1 (不互斥)

紧急位 URG=1 时, 紧急指针有效 (表示这是紧急数据, 应尽快插队发送)

推送位 PSH=1 时, 表示希望接收方尽快回复 (用于交互式通信)

复位位 RST=1 时, 表示出现严重差错 (如主机崩溃), 必须释放连接. 亦可用于拒绝非法报文段 (黑客攻击)

同步位 SYN=1 时, 表示这是一个连接请求或连接接受报文

终止位 FIN=1 时, 表示该报文段的发送方数据已发完, 要求释放传输连接

仅握手①②的SYN=1, 其他TCP报文段的SYN=0; 仅握手①②的FIN=1, 其他TCP报文段的FIN=0

窗口 (rwnd或rwnd) 16 bit, 表示接收窗口的大小, 即从本报文段首部中的ack-seq算起, 接收方还能接收多少数据 (以字节为单位)

实现“流量控制”的关键

建立TCP连接时, 在握手①②选项中协商MSS (最大段长)

MSS的值表示接下来的数据传输中, 1个TCP报文段最多携带多少数据 (首部不算在内) 不太大, 以免在IP层被分片 两边可分别设置

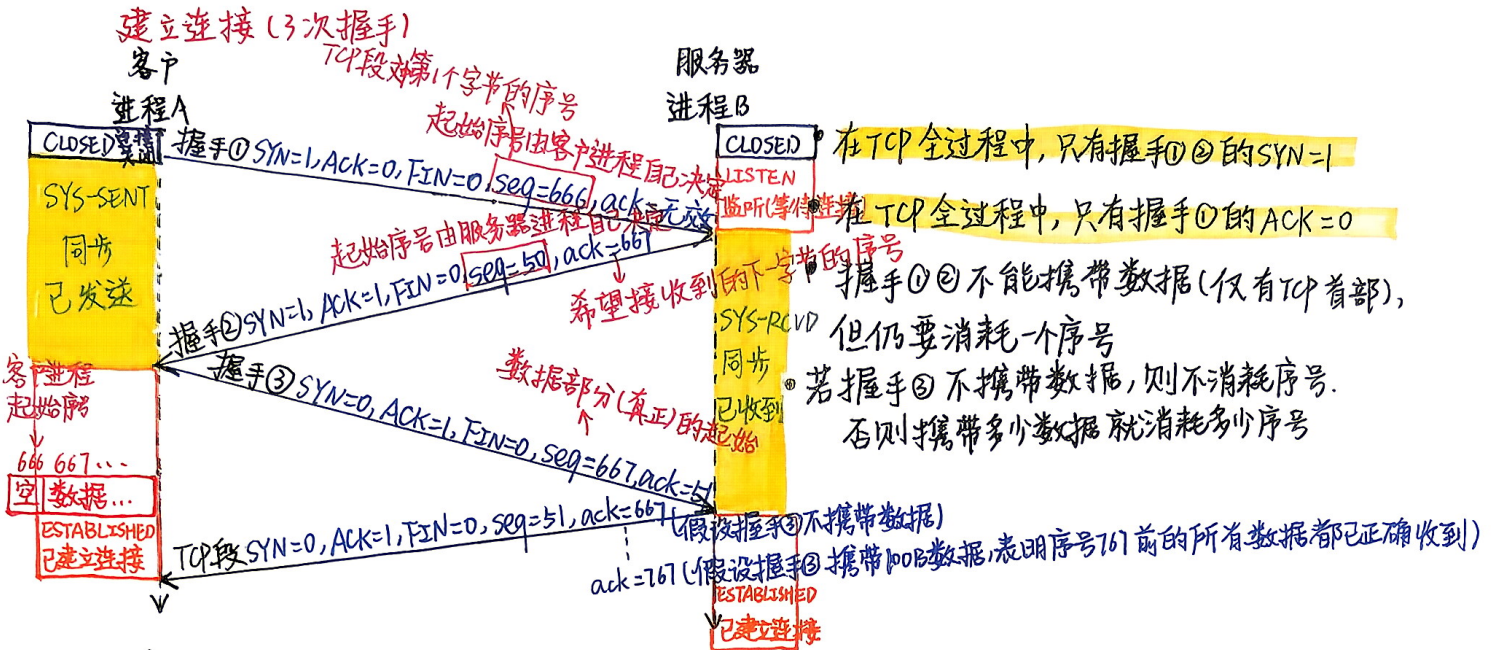
TCP连接管理

TCP连接管理

(3次握手, 4次挥手)

SYN、ACK、FIN、seq、ack-seq 的值?

每发出/收到一个握手/挥手报文段后, 进程的TCP状态转换
建立连接、断开连接的最短耗时分析



例. 主机甲向主机乙发送一个 (SYN=1, seq=11220) 的TCP段, 期望与主机乙建立TCP连接, 若主机乙接受该请求, 则主机乙向主机甲发送的正确的TCP段可能是 (C)

- A. (SYN=0, ACK=0, seq=11221, ack=11221)
- B. (SYN=1, ACK=1, seq=11220, ack=11220)
- ☒ C. (SYN=1, ACK=1, seq=11221, ack=11221)
- D. (SYN=0, ACK=0, seq=11220, ack=11220)

与题无关, 无所谓

例. 若客户首先向服务器发送FIN段请求断开TCP连接, 则当客户收到服务器发送的FIN段并向服务器发送的FIN段并向服务器发送ACK段后, 客户的TCP状态转换为 (B)

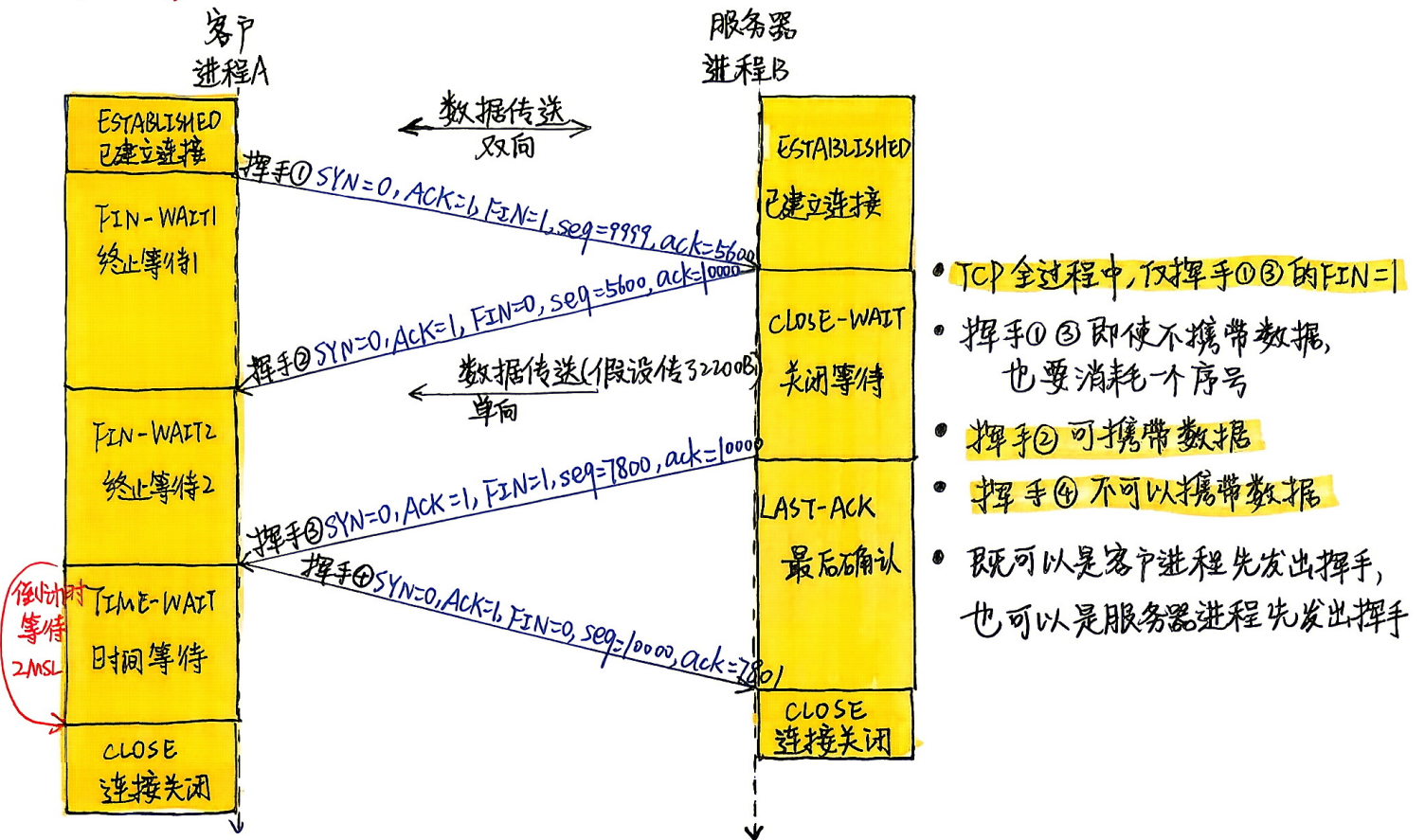
- A. CLOSE-WAIT
- ☒ B. TIME-WAIT
- C. FIN-WAIT-1
- D. FIN-WAIT-2

考察释放连接阶段的状态转换

从发出握手①, 到客户端进程可以发送数据, 至少需要多久? (收到②之后) 1 RTT

从发出握手①, 到服务器进程可以发送数据, 至少需要多久? (收到③之后) 1.5 RTT

释放连接 (4次挥手)



- 客户进程收到挥手③后立即进入 **TIME-WAIT** 状态, 并启动 "TIME-WAIT" 计时器, 倒计时 $2MSL$ 后进入 **CLOSE** 状态 (如果等待期间重复收到挥手③, 就重置计时器)
- MSL - 最长报文段寿命, 是由TCP协议规定的一个固定时间长度 (单位: s 或 ms)

MSS : 一个TCP报文段中最多可携带? 字节数据 (单位: B)

从客户发出挥手①, 到客户进程进入 **CLOSE** 状态, 至少需要多久? $1RTT + 2MSL$

从客户发出挥手①, 到服务器进程进入 **CLOSE** 状态, 至少需要多久? $1.5RTT$

- 若服务器收到挥手①时已无待传送数据, 则可连续发出挥手②③
- CLOSE-WAIT**、**FIN-WAIT2** 都极短暂, 接近0

例. 假设客户C和服务器S已建立一个TCP连接, 通信往返时间 $RTT = 50ms$,

最长报文段寿命 $MSL = 800ms$, 数据传输结束后, C主动请求断开连接.

若从C主动向S发出 **FIN** 段时刻算起, 则C和S进入 **CLOSE** 状态所需的时间至少分别是 (D)

A. $850ms, 50ms$ B. $1650ms, 50ms$ C. $850ms, 75ms$ D. $1650ms, 75ms$

$1RTT + 2MSL$

$1.5RTT$

TCP 可靠传输与流量控制

可靠传输
&
流量控制

回顾: TCP 首部相关字段 — Ack, seq, ack (ack-seq), rwnd (rwnd)

可靠传输 { 序号 — 进程在建立连接时确定起始序号, 数据传输过程中, 每字节对应一个序号
确认机制
重传机制

流量控制 { 接收方维持一个接收窗口 (rwnd)
(滑动窗口机制) 发送方维持一个发送窗口

假设客户进程

{ 发送缓冲区: 10B
接收缓冲区: 10B

关注序号、
接收窗口大小

关注累积确认

关注捎带确认

返回ack时:

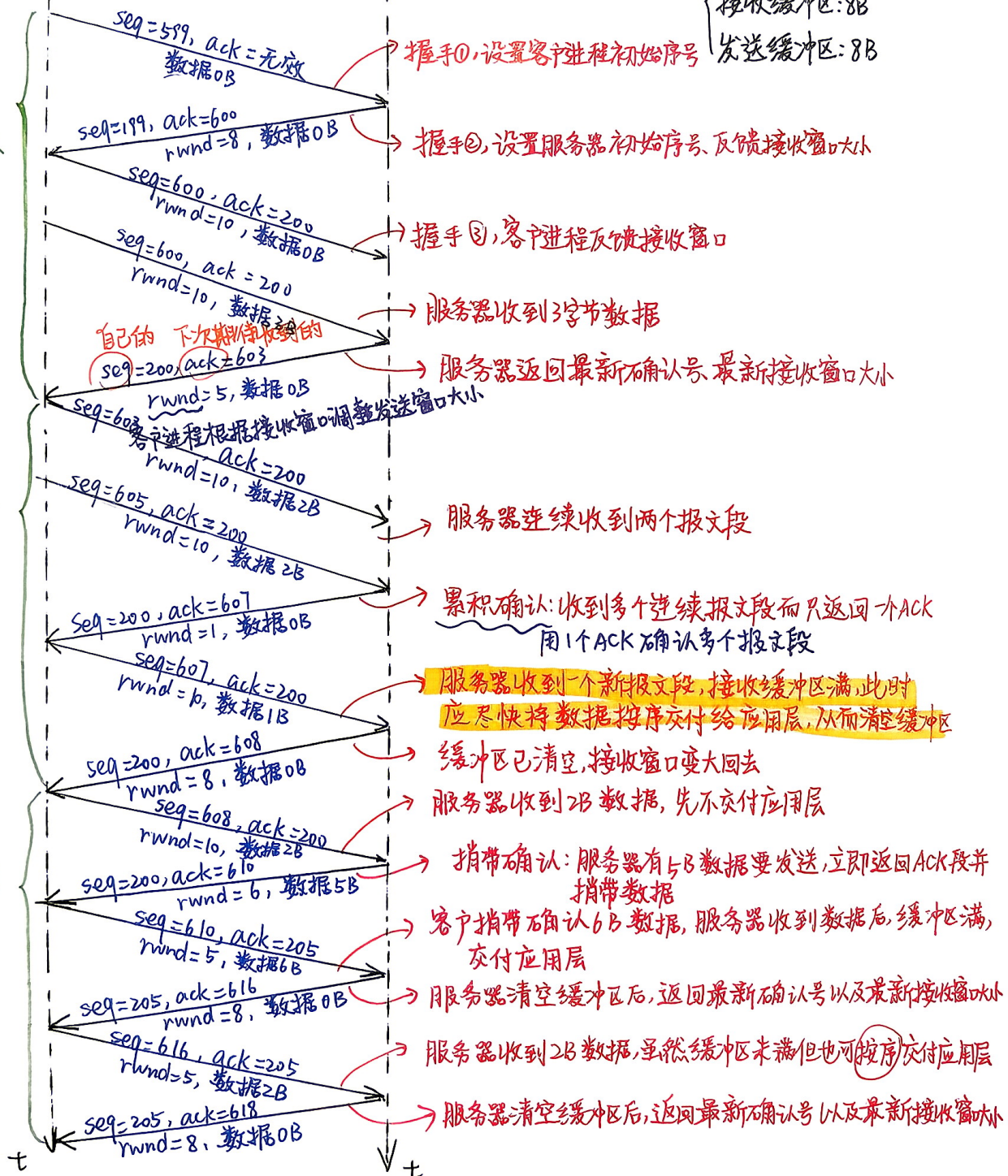
TCP 首部 数据

客户进程

服务器进程

假设服务器进程

{ 接收缓冲区: 8B
发送缓冲区: 8B

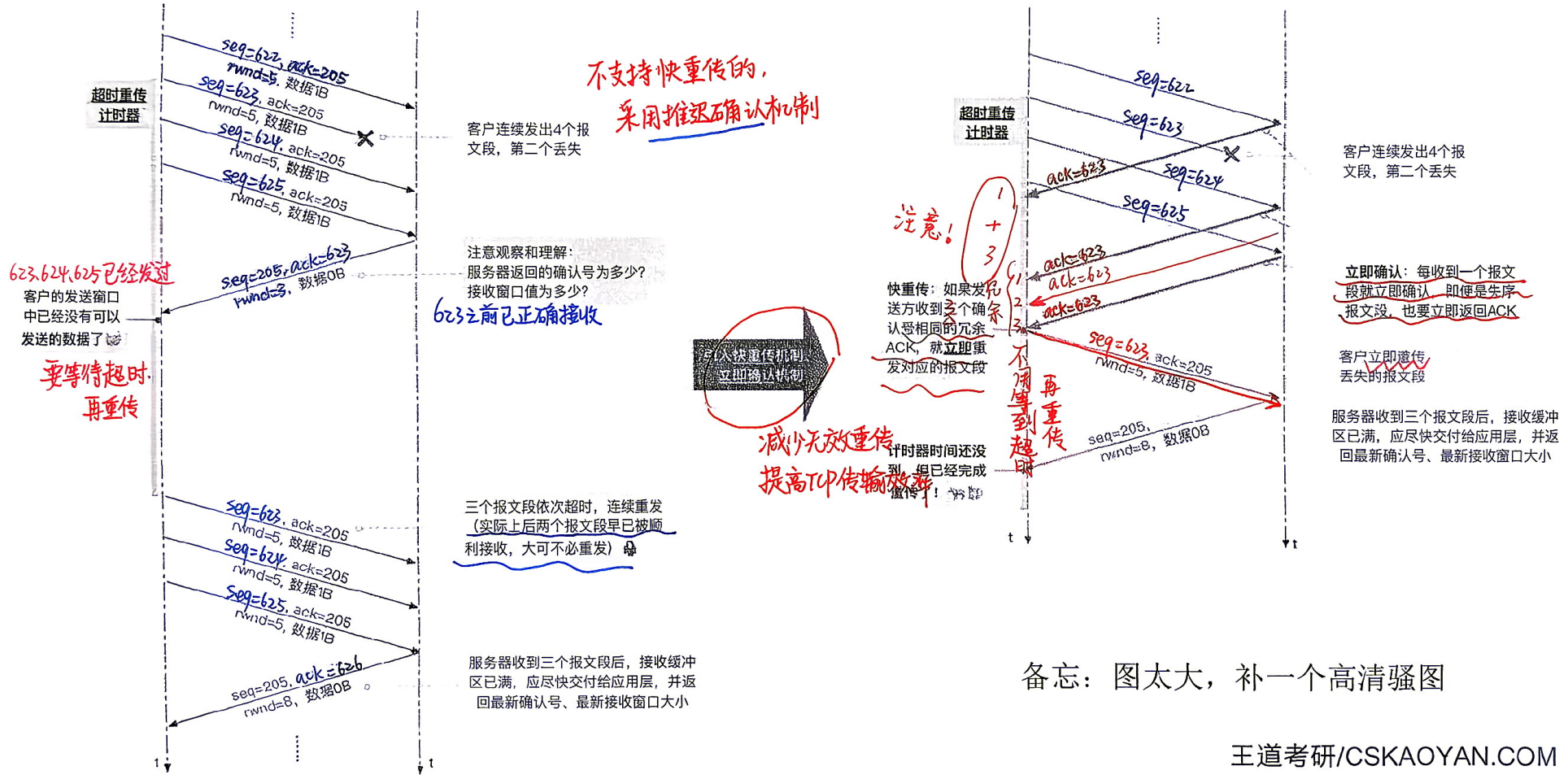


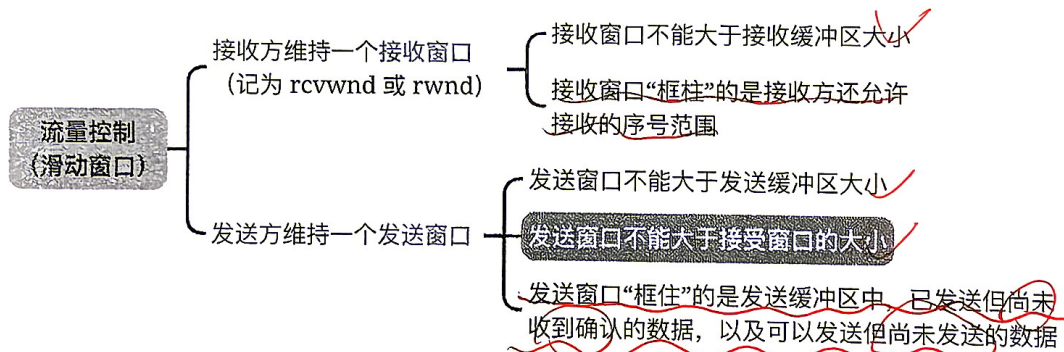
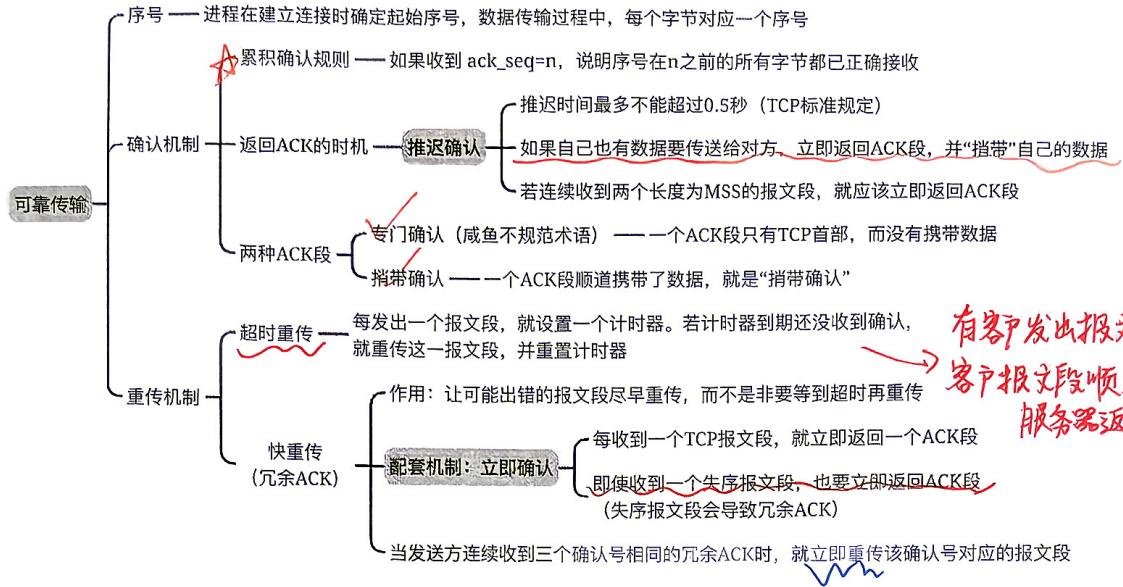
一个端口可以支持多个TCP连接, 但每个TCP连接仅支持一对一的全双工通信

TCP协议的发送缓冲区、接收缓冲区是在建立连接时由操作系统内核分配的, 大小不固定

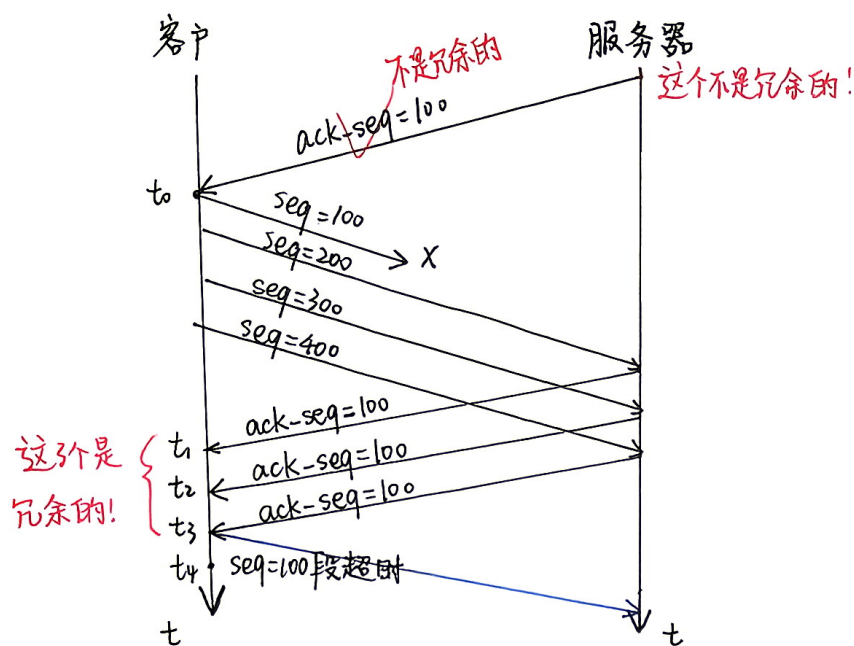
发送窗口的大小不能超过发送缓冲区的大小, 也不能超过对方接收窗口的大小

重点体会：快重传机制、立即确认机制



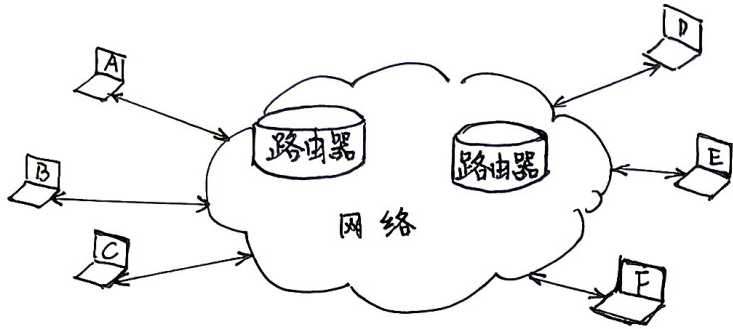


例. 某用户通过一个TCP连接向服务器发送数据的部分过程如图所示. 客户在 t_1 时刻第一次收到确认序列号 $ack-seq=100$ 的段, 并发送序列号 $seq=100$ 的段, 但发生丢失. 若TCP支持快速重传, 则客户重新发送 $seq=100$ 段的时刻是 t_3 .



拥塞控制 { 慢开始算法 → 适用于严重拥塞的情况, 可迅速调小 cwnd
 拥塞避免算法 → 适用于不拥塞的情况, 小心翼翼地调大 cwnd
 快重传机制(冗余ACK) → 快恢复算法 → 适用于有点拥塞的情况, 可适当减小 cwnd

TCP拥塞控制-TCP流量控制的区别与联系



当前网络传输数据的能力 ↑
 接收方接收数据的能力 ↑
 接收窗口 拥塞窗口

$$\text{发送窗口的上限值} = \min[rwnd, cwnd]$$

流量控制: 控制端到端的数据发送量, 是“局部的”

拥塞控制: 控制整个网络中每台主机的数据发送量, 降低路由器负载, 是“全局的”

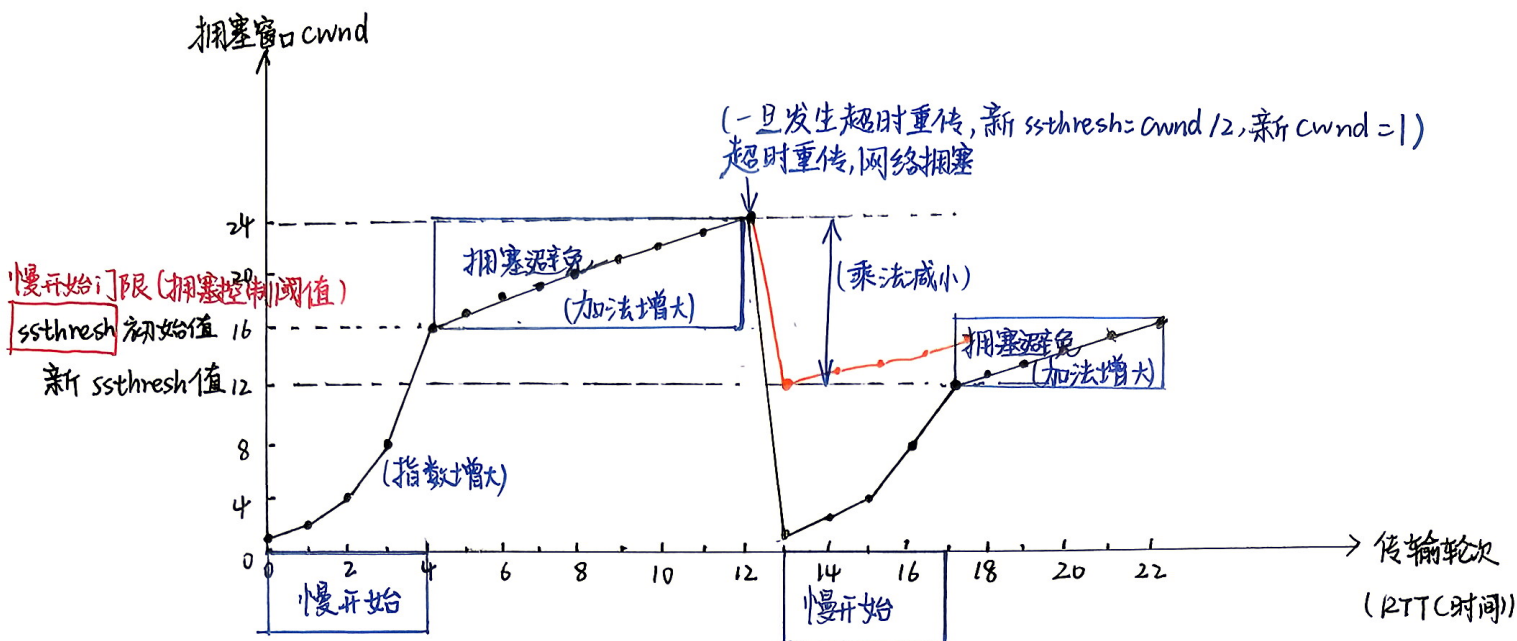
如何判断网络拥塞?

- 发出的每个报文段, 都能顺利地收到ACK确认 → 不拥塞
- 发出的报文段未能按时收到ACK, 引发超时重传 → 严重拥塞 (丢包)
- 收到冗余ACK, 引发快重传 → 有点拥塞

若检测到网络拥塞怎么办?

- ① 迅速减少发送的数据量
- ② “严重拥塞”就“迅速”缩小拥塞窗口
- ③ “有点拥塞”就“适当”缩小拥塞窗口

为聚焦拥塞控制,假设接收方的接收窗口足够大,发送窗口大小只受拥塞窗口限制

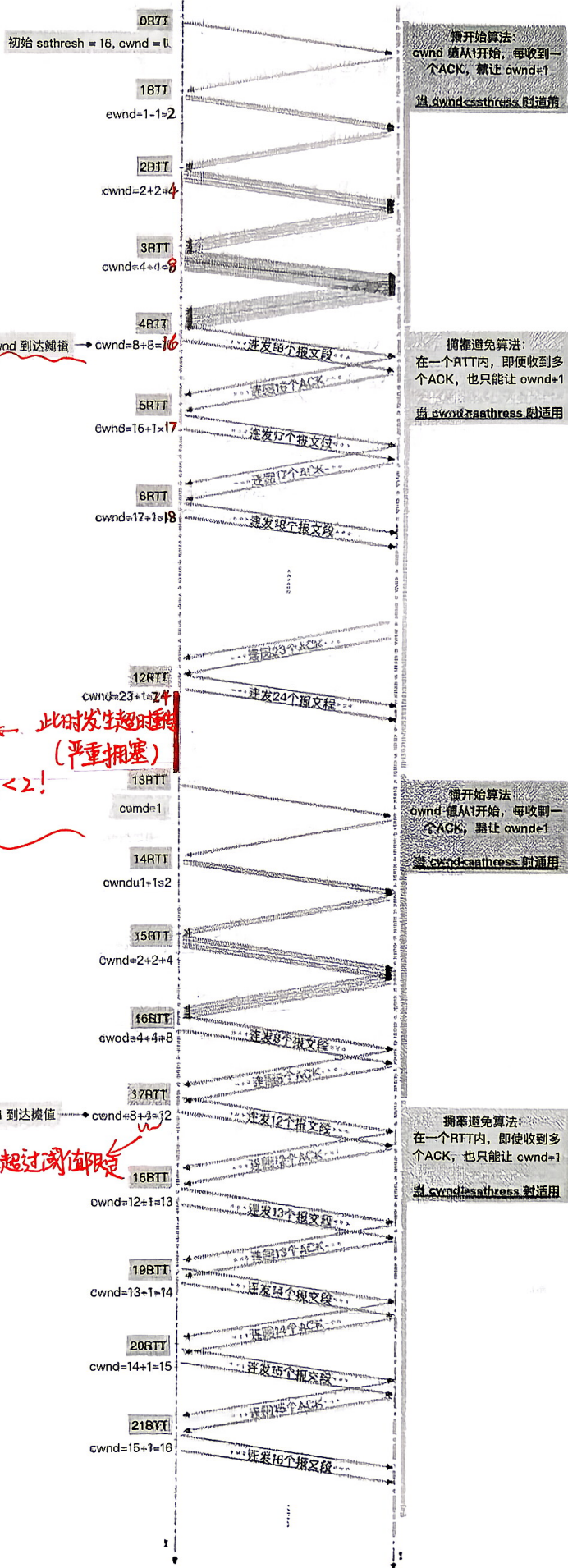


慢开始算法: cwnd 值从1开始, 每收到一个ACK就让cwnd+1 (当cwnd < ssthresh时适用)

拥塞避免算法: 在1个RTT内,即使收到多个ACK,也只让 $cwnd+1$ (当 $cwnd \geq ssthresh$ 时适用)

快重传：当发送方收到3个确认号相同的冗余ACK时，立即重传对应报文段

快恢复算法: 一旦发生快重传, $\text{new ssthresh} = \text{new cwnd} = \text{cwnd} / 2$, 然后切换到拥塞避免算法(加法增大)



$ssthresh = cwnd / 2 = 12$ ← 此时发生超时重传 (严重拥塞)
cwnd = 1
注意: ssthresh 不能 < 2!

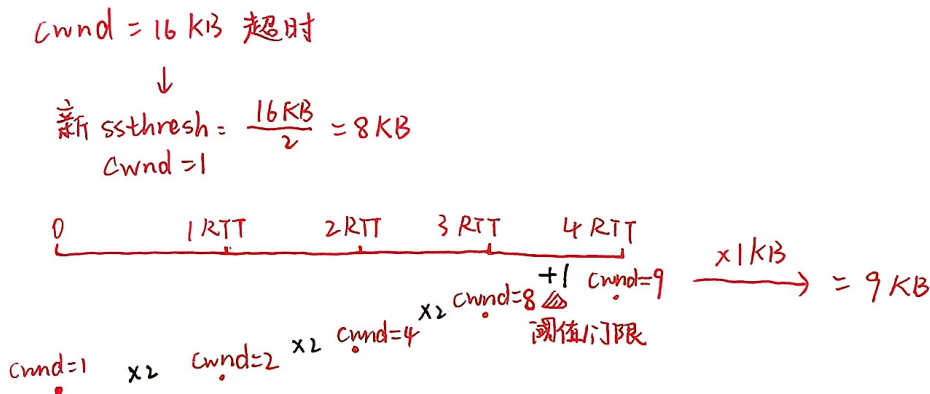
不能超过阈值限定

题目特点:

- 常与“流量控制”综合考察,此时需要考虑接收窗口的大小
- 通常只涉及单向传输,即TCP连接双方只有一方发送数据
- 通常默认每个TCP报文段都以最大段长MSS满载数据
- 拥塞窗口的大小常以“MSS的倍数”作为单位
- 接收方收到一个报文段,会“立即确认”而不是“推迟确认”
这样才有冗余ACK,快重传

例. 1个TCP连接总以 1KB 的最大段长 ^{MSS (若是2KB,结果是18KB)} 发送TCP段,发送方有足够多的数据要发送.

当拥塞窗口为16KB时发生超时,若接下来4个RTT(往返时间)时间内的TCP段的传输都成功,那么当第4个RTT时间内发送的所有TCP段都得到肯定应答时,拥塞窗口大小是 9KB.



例. 主机甲和乙新建一个TCP连接,甲的拥塞控制初始阈值为32KB,甲向乙始终以MSS=1KB大小的段发送数据,并一直有数据发送;乙为该连接分配16KB接收缓存,并对每个数据段进行确认,忽略段传输延迟.若乙收到的数据全存入缓存,不被取走,则甲从连接建立成功时刻起,未发送超时情况下,经过4个RTT,甲的发送窗口是 1KB (大小).

